

PROGRAMACIÓN AVANZADA

Base de datos – Parte III



Ing. Matias Macrino

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Consultas preparadas:**

Se basa en consultas SQL precompiladas. La ventaja que nos brinda es utilizar parámetros en lugar de valores fijos.

▶ Dichas consultas son gestionadas bajo la interfaz ***PreparedStatement***.

▶ La idea es precompilar una ***sentencia SQL***, utilizando parámetros en vez de valores fijos.

▶ Logrando sustituir los parámetros por valores concretos en el momento que se desea ejecutar la sentencia.

▶ Para la creación de una consulta se utiliza el método ***prepareStatement()*** de la interfaz ***Connection***. Este método recibe como parámetro la consulta SQL para su compilación.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

```
Connection con = null;
PreparedStatement ps = null;
try {
    Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
} catch (ClassNotFoundException e) {
    e.printStackTrace();
}

try {
    con = DriverManager.getConnection("jdbc:sqlserver://127.0.0.1:1433;databaseName=Facos", "sa", "123456");
} catch (SQLException e) {
    e.printStackTrace();
}

try{
    String strSQL = "UPDATE Clientes SET clie_razon = ? WHERE clie_id = ?";
    ps = con.prepareStatement(strSQL);
} catch (SQLException e) {
    e.printStackTrace();
}

try {
    ps.close();
    con.close();
} catch (SQLException e) {
    e.printStackTrace();
}
```

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

```
try{
    String strSQL = "UPDATE Clientes SET clie_razon = ? WHERE clie_id = ?";
    ps = con.prepareStatement(strSQL);
}catch(SQLException e){
    e.printStackTrace();
}
```

- Los valores de los campos se indican mediante el símbolo “?” en la sentencia SQL para su posterior sustitución.
- El símbolo “?” hace referencia a un valor, independientemente de su tipo, por lo cual no es necesario encerrar este símbolo entre comillas en el caso de tratarse de texto.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Asignación de parámetros:**

- ▶ Una vez creada, la consulta está, como su nombre indica, preparada para ser ejecutada tantas veces como se requiera.
- ▶ Antes de ejecutar la sentencia, es necesario asignar los valores a los parámetros definidos en la sentencia.
- ▶ El método que se utiliza para ingresar los valores recibe dos parámetros. Primero recibe el índice de la posición de la sentencia SQL y segundo el valor a ingresar.
- ▶ Ejemplo:
 - ▶ `void setXXX(int índice_parámetro, xxx valor).`
 - ▶ Siendo XXX el tipo de dato primitivo de Java mas Date, Object y String.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ *Asignación de parámetros:*

```
try{
    String strSQL = "UPDATE Clientes SET clie_razon = ? WHERE clie_id = ?";
    ps = con.prepareStatement(strSQL);
    ps.setString(1, "PILAR EVENTOS S.A");
    ps.setInt(2, 2);
}catch(SQLException e){
    e.printStackTrace();
}
```

- Para este ejemplo podemos observar que:
 - ✓ Para valores de texto en la sentencia no es necesario poner las " en las sentencia.
 - ✓ Para valores de texto se asigna el parámetro a través del método ***setString(int, String)***
 - ✓ Para valores numéricos se asigna el parámetro a través del método ***setInt(int, int)***

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Métodos `execute()`, `executeQuery()` y `executeUpdate()`:**

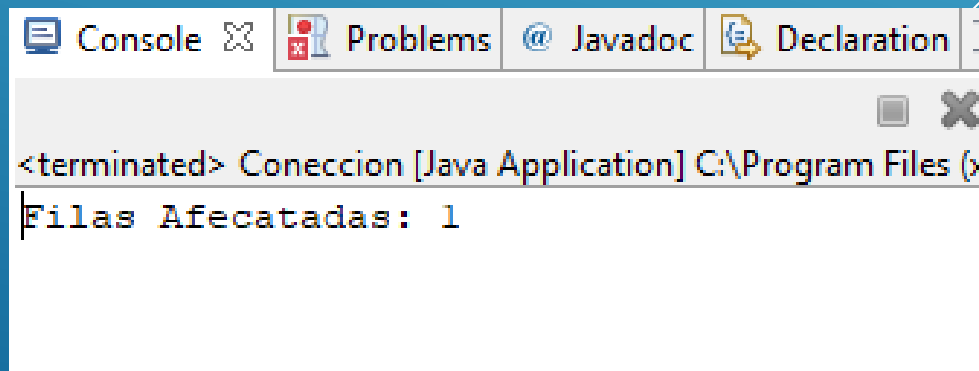
- ▶ Son sobrecargados y en esta versión, no toman ningún tipo de argumentos, de esta forma, a estos métodos nunca se les deberá pasar por parámetro el objeto String que representaba la sentencia SQL a ejecutar.
- ▶ En este caso, un objeto **`PreparedStatement`** ya es una sentencia SQL por si misma, a diferencia de lo que ocurría con las sentencias **`Statement`** que poseían un significado solo en el momento que se ejecutaban.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ Métodos execute(), executeQuery() y executeUpdate():

```
try{
    String strSQL = "UPDATE Clientes SET clie_razon = ? WHERE clie_id = ?";
    ps = con.prepareStatement(strSQL);
    ps.setString(1, "PILAR EVENTOS S.A");
    ps.setInt(2, 2);
    int filasAfectadas = ps.executeUpdate();
    System.out.println("Filas Afecatas: " + filasAfectadas);
}catch(SQLException e){
    e.printStackTrace();
}
```



JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Interfaz CallableStatement:**

▶ Dicha interfaz ofrece la posibilidad de manejar parámetros de salida y de realizar llamadas a **procedimientos (stored procedures)**. Estos últimos son altamente performantes.

▶ La sintaxis para el llamado a un **SP** es la siguiente: **{nombre_SP[(?, ?,....)]}**

▶ Ejemplo de llamada a un SP sin parámetros:

{nombre_SP}
{SP_Consulta}

▶ Para instanciar un objeto de la clase que implementa esta interfaz se debe ejecutar el método **prepareCall** sobre la conexión activa. El parámetro de este método es el SP almacenado a ejecutar.

CallableStatement inst = conexion.prepareCall("{SP_Consulta}");

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Interfaz CallableStatement:**

- ▶ Una sentencia **CallableStatement** puede tener parámetros de entrada, salida y de entrada/salida
- ▶ Para pasarle parámetros de entrada se utilizan los métodos **setXXX** que hereda de la interfaz **PreparedStatement**.
- ▶ Si el SP devuelve parámetros de salida, cada parámetro debe de ser registrado antes de ejecutar el objeto **CallableStatement**, con el método **registerOutParameter** sobre el objeto **CallableStatement** definido con dos parámetros, valor ordinal del parámetro y el tipo de dato retornado.
- ▶ Luego de ejecutar la sentencia, se pueden recuperar con los métodos **getXXX**
Ejemplo parámetros de entrada: **inst.setInt(1, 200);**
Ejemplo parámetros de salida: **inst.registerOutParameter(1, java.sql.Types.INTEGER);**
- ▶ El SP se debe de ejecutar mediante el método **execute();**.
Ejemplo: **inst.execute();**

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Ejecución de sentencias SQL:

▶ **Interfaz CallableStatement:**

```
try {
    CallableStatement cst = null;
    cst = con.prepareCall("EXEC SP_Acciones_Articulos ?, ?, ?, ?, ?, ?");
    cst.setString(1, "Articulo 1");
    cst.setString(2, "Articulo descripcion nuevo");
    cst.setInt(3, 5);
    cst.setInt(4, 1);
    cst.setString(5, "A");
    cst.registerOutParameter(6, Types.VARCHAR);

    cst.execute();
    System.out.println(cst.getString("resultado"));

} catch (SQLException e) {
    e.printStackTrace();
}
```

JDBC – JAVA DATABASE CONNECTIVITY

▣ API JDBC – Transacciones:

- ▣ Hay veces que necesitamos que algunas sentencias SQL tomen efectos a menos que otras se completen.
- ▣ Las transacciones son un conjunto de ordenes que representan la unidad de trabajo mínima e indivisible (acción atómica).
- ▣ Los gestores de BD proveen mecanismos necesarios para construir una transacción.
- ▣ Lo mismo podemos simular en una aplicación Java. Sin depender de los mecanismos propios del motor de BD.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC – Transacciones:

- ▶ Un claro ejemplo de una transacción, es la de transferir un cierto monto de una cuenta bancaria A (origen) a otra cuenta bancaria B (destino).
- ▶ Una vez de tener realizadas correctamente todas las sentencias SQL se utiliza el método ***commit()*** de la clase ***Connection***
- ▶ En el caso de que haya fallado alguna de la sentencias SQL se utiliza el método ***rollback()*** de la clase ***Connection***

Try {

```
con.setAutoCommit(false);  
... Bloque de sentencias SQL ...  
con.commit();  
} catch(SQLException e) {  
con.rollback();  
}
```

SQL Server Types	JDBC Types (java.sql.Types)	Java Language Types
bigint	BIGINT	long
binary	BINARY	byte[]
bit	BIT	boolean
char	CHAR	String
date	DATE	java.sql.Date
datetime	TIMESTAMP	java.sql.Timestamp
datetime2	TIMESTAMP	java.sql.Timestamp
datetimeoffset (2)	microsoft.sql.Types.DATETIMEOFFSET	microsoft.sql.DateTimeOffset
decimal	DECIMAL	java.math.BigDecimal
float	DOUBLE	double
image	LONGVARBINARY	byte[]
int	INTEGER	int
money	DECIMAL	java.math.BigDecimal
nchar	CHAR NCHAR (Java SE 6.0)	String
ntext	LONGVARCHAR LONGNVARCHAR (Java SE 6.0)	String
numeric	NUMERIC	java.math.BigDecimal
nvarchar	VARCHAR NVARCHAR (Java SE 6.0)	String
nvarchar(max)	VARCHAR NVARCHAR (Java SE 6.0)	String
real	REAL	float
smalldatetime	TIMESTAMP	java.sql.Timestamp
smallint	SMALLINT	short
smallmoney	DECIMAL	java.math.BigDecimal
text	LONGVARCHAR	String
time	TIME (1)	java.sql.Time (1)
timestamp	BINARY	byte[]
tinyint	TINYINT	short
udt	VARBINARY	byte[]
uniqueidentifier	CHAR	String
varbinary	VARBINARY	byte[]
varbinary(max)	VARBINARY	byte[]
varchar	VARCHAR	String
varchar(max)	VARCHAR	String
xml	LONGVARCHAR LONGNVARCHAR (Java SE 6.0)	String SQL XML

PROGRAMACIÓN AVANZADA

Patrones de Diseño



Ing. Matias Macrino

PATRONES DE DISEÑO

▶ Patrones creacionales:

▶ Singleton:

-> Garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global.

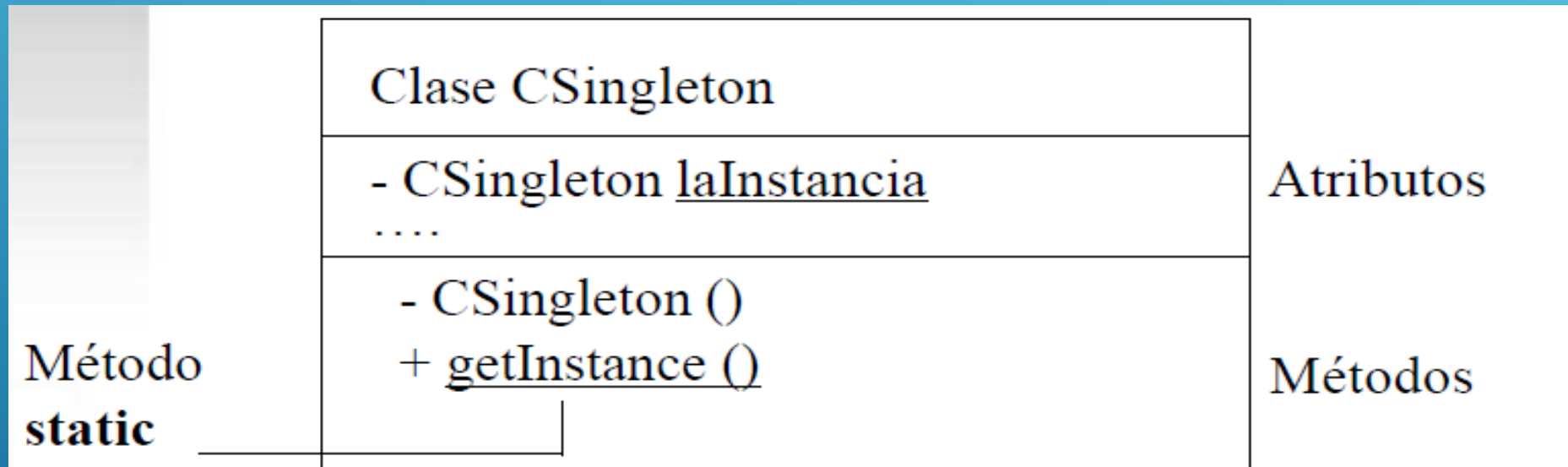


-> Esta diseñado para restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto.

-> Es necesario cuando hay clases que tienen que gestionar de manera centralizada un recurso.

PATRONES DE DISEÑO

► Patrones creacionales Singleton ejemplo:



► El constructor de la clase *debe ser privado*.

► Se proporciona un método **Static** en la clase que devuelve “*La única instancia de la clase: getInstance()*”

PATRONES DE DISEÑO

▶ Ejemplo Singleton:

```
public class ObjetoUnico {  
  
    private String nombreObjeto;  
    private static ObjetoUnico objetoUnico;  
  
    private ObjetoUnico(String nombreObjeto) {  
        this.nombreObjeto = nombreObjeto;  
    }  
  
    public static ObjetoUnico getInstance(String nombreObjeto) {  
        if(objetoUnico == null) {  
            objetoUnico = new ObjetoUnico(nombreObjeto);  
        }  
        return objetoUnico;  
    }  
  
    public String getNombreObjeto() {  
        return this.nombreObjeto;  
    }  
  
}
```

PATRONES DE DISEÑO

▶ Ejemplo Singleton:

```
public static void main(String[] args) {  
  
    ObjetoUnico objetoUno = ObjetoUnico.getInstance("Silla");  
    ObjetoUnico objetoDos = ObjetoUnico.getInstance("Mesa");  
  
    System.out.println(objetoUno.getNombreObjeto());  
    System.out.println(objetoDos.getNombreObjeto());  
}
```

